Reviewer Pack — Möbius Function of Minterm Lattice Bounds Indexing-Lifted Co…

Entry b63c45216b7a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-27 08:47:05 UTC

Möbius Function of Minterm Lattice Bounds Indexing-Lifted Communication

Entry ID: `b63c45216b7a` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-04-27 08:47:05 UTC

1. Conjecture

Field A (mathematical branch): Topological combinatorics (Möbius function and order-complex Euler characteristic on the meet-closure lattice of minterm supports, in the spirit of Rota and Stanley)
Field B (complexity object): Deterministic two-party communication complexity of Indexing-gadget-lifted monotone functions (Göös–Pitassi–Watson lifting), with deterministic decision-tree depth as the lifted-side proxy

Statement:

> Let f:{0,1}^n→{0,1} be monotone with prime-implicant supports S_1,…,S_k⊆[n]; let L_f be the meet-closure lattice generated by {S_i} together with an adjoined top 1̂=[n] and bottom 0̂=∅, and let μ_f:=|μ_{L_f}(0̂,1̂)| be its Möbius number. Then for all n≤20 the deterministic decision-tree depth satisfies D^dt(f) ≥ ⌈log₂(1+μ_f)⌉, and consequently the Göös–Pitassi–Watson Indexing-gadget lifting yields D^cc(f∘IND_b) = Ω((log b)·log₂(1+μ_f)). A single monotone f with ⌈log₂(1+μ_f)⌉ > D^dt(f) refutes the conjecture.

Rationale (proposer's reasoning):

> Lifting theorems convert decision-tree depth to communication, so any combinatorial invariant that lower-bounds D^dt(f) automatically lower-bounds lifted communication. The Möbius number of the minterm closure lattice measures how much non-Boolean-lattice structure the accepting set carries; intuitively, each query in a decision tree can only halve the alternating Möbius sum on a chain, so depth must dominate log of |μ|. This bridges topological combinatorics to communication lower bounds, a pairing with no prior occurrences I am aware of.

Taxonomy category: `LIFTING` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `2246a8c4a563b38d`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across ~400 random monotone DNFs (n∈{6,8,10,12}, k∈{3..8}, implicant size 2–4), the conjecture is SUPPORTED iff ≥99% of instances satisfy D^dt(f) ≥ ⌈log₂(1+μ_f)⌉ AND the median slack D^dt(f) − ⌈log₂(1+μ_f)⌉ is ≥1. Any single instance with ⌈log₂(1+μ_f)⌉ > D^dt(f) FALSIFIES.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.82	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `Möbius function monotone Boolean decision tree complexity prime implicant lattice` - `Göös Pitassi Watson indexing gadget lifting monotone communication complexity` - `order complex Euler characteristic decision tree lower bound monotone function`

Top relevant hits considered: - [http://arxiv.org/abs/0710.4339v1] Heavy-Quark Masses from the Fermilab Method in Three-Flavor Lattice QCD - [http://arxiv.org/abs/1111.0981v1] Form factors for B to Kll semileptonic decay from three-flavor lattice QCD - [http://arxiv.org/abs/1810.08668v2] Parity Decision Tree Complexity is Greater Than Granularity - [http://arxiv.org/abs/1605.01142v2] Separations in communication complexity using cheat sheets and information complexity - [http://arxiv.org/abs/2001.02144v1] Lifting with Simple Gadgets and Applications to Circuit and Proof Complexity - [http://arxiv.org/abs/1703.07666v1] Query-to-Communication Lifting for BPP - [http://arxiv.org/abs/1801.08709v2] Adaptive Lower Bound for Testing Monotonicity on the Line - [http://arxiv.org/abs/2512.11833v1] Soft Decision Tree classifier: explainable and extendable PyTorch implementation

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
import json
from sys import argv

def generate_random_dnf(n, k, implicant_size):
    variables = list(range(n))
    implicants = []

    while len(implicants) < k:
        implicant = set()
        for _ in range(implicant_size):
            var = random.choice(variables)
            if random.choice([True, False]):
```

```
                implicant.add(var)
            else:
                implicant.add(-var)

        if all(len(set(a).intersection(b)) == 0 for b in implicants):
            implicants.append(frozenset(implicant))

    return implicants

def recursive_minimum(truth_table, variables, memo):
    if len(variables) == 1:
        var = variables[0]
        return min(truth_table[var][True], truth_table[var][False])

    var = variables[0]
    memo_key = (var, tuple(sorted(variables[1:])))
    if memo_key in memo:
        return memo[memo_key]

    true_case = recursive_minimum(truth_table, variables[1:], memo)
    false_case = recursive_minimum(truth_table, variables[1:], memo)
    result = min(true_case, false_case)
    memo[memo_key] = result
    return result

def compute_möbius_number(lattice):
    n = len(lattice)
    mu = [0] * (2 ** n)
    mu[0] = 1

    for i in range(1, 2 ** n):
        for j in range(i):
            if lattice[j].issubset(i):
                mu[i] -= mu[j]

    return abs(mu[-1])

def run_trial(seed: int) -> dict:
    random.seed(seed)
    results = []

    for n in [6, 8, 10, 12]:
        for k in range(3, 9):
            for implicant_size in [2, 3, 4]:
                dnf = generate_random_dnf(n, k, implicant_size)

                truth_table = {var: {True: 0, False: 0} for var in range(n)}
                for implicant in dnf:
                    for assignment in range(1 << n):
                        if all((assignment >> i) & 1 == (abs(var) - 1) % 2 for var in implicant):
                            truth_table[implicant] += 1

                memo = {}
                depth = recursive_minimum(truth_table, list(range(n)), memo)

                lattice = [frozenset()]
```

```
                while True:
                    new_elements = set()
                    for element in lattice:
                        for i in range(n):
                            if (element | {i}) not in lattice:
                                new_elements.add(element | {i})
                    if not new_elements:
                        break
                    lattice.update(new_elements)

                mu_f = compute_möbius_number(lattice)
                required_depth = math.ceil(math.log2(1 + mu_f))

                results.append({
                    "n": n,
                    "k": k,
                    "implicant_size": implicant_size,
                    "depth": depth,
                    "mu_f": mu_f,
                    "required_depth": required_depth
                })

    total_depth = sum(result["depth"] for result in results)
    mean_depth = total_depth / len(results)
    std_dev = math.sqrt(sum((result["depth"] - mean_depth) ** 2 for result in results) / len(resul
    support_fraction = sum(1 for result in results if result["depth"] >= result["required_depth"])

    conjecture_holds = support_fraction >= 0.99 and all(result["depth"] - result["required_depth"]
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "Depth",
        "metric_value": mean_depth,
        "instances_tested": len(results),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = list(map(int, argv[1:])) if argv[1:] else [11, 23, 37, 53, 71]

    for seed in seeds:
        result = run_trial(seed)
        print(json.dumps({"SEED": seed, **result}))

    total_depth = sum(result["depth"] for result in results)
    mean_depth = total_depth / len(results)
    std_dev = math.sqrt(sum((result["depth"] - mean_depth) ** 2 for result in results) / len(resul
    support_fraction = sum(1 for result in results if result["depth"] >= result["required_depth"])

    if support_fraction >= 0.99 and all(result["depth"] - result["required_depth"] >= 1 for result
        print(f"RESULT: SUPPORTED mean={mean_depth} std={std_dev} support_fraction={support_fracti
    elif any(result["depth"] > result["required_depth"] for result in results):
        first_failing_seed = seeds[next(i for i, result in enumerate(results) if result["depth"] >
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_f
    else:
```

```
        print("RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d6637e14.py", line 125, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d6637e14.py", line 71, in run_trial
    dnf = generate_random_dnf(n, k, implicant_size)
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d6637e14.py", line 31, in generate_rand
    if all(len(set(a).intersection(b)) == 0 for b in implicants):
       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d6637e14.py", line 31, in <genexpr>
    if all(len(set(a).intersection(b)) == 0 for b in implicants):
                   ^
NameError: name 'a' is not defined
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed with a NameError in generate_random_dnf (undefined variable 'a' in a generator expression) before any instances were evaluated, so no data was produced for either the support or falsification criterion. | next: Fix the generator-expression bug in generate_random_dnf (likely should iterate over candidate 'a' against existing implicants 'b') and rerun the 400-instance sweep.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	57680
2	preregistration	claude_max	opus	0	0	8177
3	novelty	claude_max	opus	0	0	3210
4	novelty	claude_max	opus	0	0	9277
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18903
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13572
7	judge	claude_max	opus	0	0	4499

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 115318 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/b63c45216b7a.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b63c45216b7a.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/b63c45216b7a.tar.gz` (if generated)